Presentable Debugging with LLVManim

Benjamin Houle
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Benjamin_Houle@student.uml.edu

Ebenezer Oloyede
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Ebenezer_Oloyede@student.uml.edu

Lily Grippo
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Lily_Grippo@student.uml.edu

Mohammad Ahmed
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
ahmedzubair16dec@gmail.com

***Abstract*—LLVManim is a code visualization tool that generates aesthetically pleasing animations of control flow and memory operations from LLVM IR. Over the course of a single semester, we designed and implemented a three-layer pipeline—Ingestion, Transformation, and Rendering—that parses `.ll` files with `LLVMLite`, derives a typed intermediate scene graph, and renders animations through Manim Community Edition. The finished tool supports three animation modes (basic stack view, rich IR spotlight, and a three-column SSA panel), a CFG traversal animation driven by Graphviz layout, and machine-readable JSON and DOT export. The implementation comprises 26 Python modules validated by 302 automated tests at 84% line coverage, enforced by a four-stage quality gate (Ruff, Pyright, import-linter, pytest). LLVManim demonstrates that presentation-quality program visualization can be made accessible through a simple command-line interface, eliminating the need for manual Manim scripting or post-processing of LLVM's utilitarian graph output.**

CCS Concepts: **• Human-centered computing → Visualization → Visualization systems and tools → Visualization toolkits; • Software and its engineering → Software notations and tools → Compilers; • Applied computing → Education → Interactive learning environments**

***Index Terms*—program visualization, code animation, LLVM, static analysis, control flow graphs, memory visualization, educational tools, Manim, compiler infrastructure, presentation tools**

I. Introduction

Despite static analysis and data visualization being among the oldest concepts in computer science, disappointingly few graphic designers have been significantly interested in the former. While plenty of tools exist for creating control flow graphs or visualizing the stack, the majority of these favor a "function over form" paradigm that focuses on creating lightweight, readable output, assuming the user is more concerned with the information than its presentation. However, this assumption does not always hold: presenting in any context, such as at a conference or school, generally favors quickly digestible and visually appealing content over static, large blocks of text. In these cases, the presenter is forced to choose between using the unappealing generated visualization and the slog of creating something presentable for themselves.

We developed LLVManim to address this problem. By pairing LLVM's powerful static analysis infrastructure with the Manim animation library, the tool automatically generates professional-quality visualizations from LLVM Intermediate Representation (IR) and runtime traces. Through a simple command-line interface, users can produce animated stack walkthroughs, IR spotlight views with Static-Single-Assignment (SSA) value tracking, Control Flow Graph (CFG), which represents the execution paths between basic blocks in a program, traversal animations, and machine-readable scene graph exports—all without writing any Manim code or manually post-processing compiler output.

The remainder of this paper describes the tools and techniques that informed our design (Section II), the system architecture (Section III), potential risks and their mitigation (Section IV), the results of our implementation (Section V), lessons learned (Section VI), and conclusions (Section VII).

II. Related Work

Three prior projects directly shaped LLVManim's design: PythonTutor's step-by-step execution visualization, LLVM's existing analysis infrastructure and its output limitations, and the Manim animation library that serves as our rendering backend. Each is examined in turn.

A. PythonTutor—Web-Based Code Execution Visualizer

PythonTutor [1] is a widely used educational tool for visualizing program execution, originally developed to help novices build accurate mental models of how code runs. Its core contribution is a step-by-step execution trace—illustrated in Fig. 1—that exposes stack frames, heap objects, and control flow in a concrete, visual manner. As described by its creator, Philip J. Guo, PythonTutor aims to "help people overcome a fundamental barrier to learning programming: understanding what happens as the computer runs each line of code" [2].

PythonTutor has been shown to be effective in improving students' understanding of programming concepts; Guo's further studies demonstrated significant learning gains when

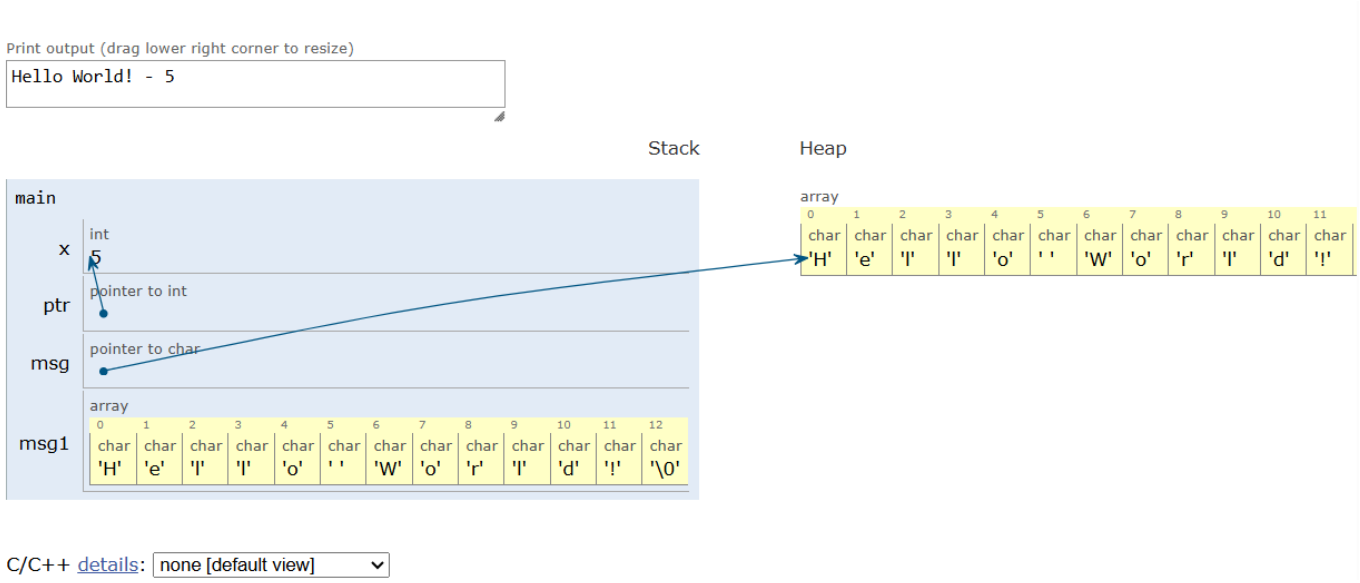


Fig. 1: Example PythonTutor.com output showing a step-by-step execution trace with stack frames and heap objects.

PythonTutor was used as a supplement to traditional instruction [3]. This demonstrates the educational value of connecting code execution to visual representations, as well as the benefits of an accessible, interactive tool.

However, the visualizations produced by PythonTutor are relatively basic and not suitable for presentation purposes. PythonTutor's primary focus is on clarity and educational value rather than aesthetics, resulting in output that may be difficult to read or visually unappealing when projected to an audience. This, along with the fact that the tool is web-based and not designed for offline use, makes it less than ideal for use in settings where visual impact is important, such as conference presentations. PythonTutor is additionally restricted by its limited customization options, minimal library of visualization methods, and support for only C, C++, Java, Javascript, and Python.

B. LLVM Analysis and Transform Passes

LLVM has emerged as the de-facto standard compiler infrastructure for modern software development, powering everything from the development environment for Apple operating systems to cutting-edge systems research efforts. Its adoption by the compiler community can be attributed to several key strengths—a modular architecture that separates language frontends from optimization backends, an extensive suite of analysis and transformation passes that expose deep insights into program behavior, and robust support for over 30 programming languages, including those supported by PythonTutor. LLVM even provides users with an API to define their own language [4].

At the core of LLVM's value proposition for code analysis is its Intermediate Representation, a low-level and platform-independent format that captures program semantics in a standardized way. This IR serves as the foundation for powerful built-in analysis passes that can generate control flow graphs, dominator trees, data flow analyses, and detailed memory access patterns. For compiler developers and researchers, these capabilities are invaluable—LLVM effectively solves the hard problem of extracting structured information from code, allowing them to focus on higher-level tasks like optimization or verification [5].

However, LLVM's visualization capabilities tell a different story. The standard approach for visualizing analysis results relies on generating DOT[1] files through passes like `-dot-cfg` or `-view-cfg`, which are then rendered using Graphviz [6]. While functionally correct, the resulting diagrams—an example of which is shown in Fig. 2—embody a strictly utilitarian aesthetic: monochrome nodes and edges, densely packed labels filled with LLVM IR instructions, and layouts optimized for completeness rather than clarity. This "function over form" philosophy serves well enough as a developer tool for debugging optimizer passes, but it fails as a solution for communication beyond that narrow domain.

The barrier extends beyond mere aesthetics. Interpreting these diagrams requires familiarity with LLVM IR syntax—understanding what "`%4 = load i32, i32* %3`"[2] means demands compiler knowledge that most audiences lack. The visualizations are fundamentally static images with no mechanism for progressive disclosure or animation, forcing presenters to mentally simulate execution flow while pointing at a frozen graph. Moreover, customization options are severely limited; changing colors, adjusting layouts, or highlighting specific execution paths requires manual post-

[1] A plain-text graph description language used by Graphviz to define nodes and edges for rendering graphs.

[2] This instruction loads a 32-bit integer from the memory address stored in `%3` into the register `%4`.

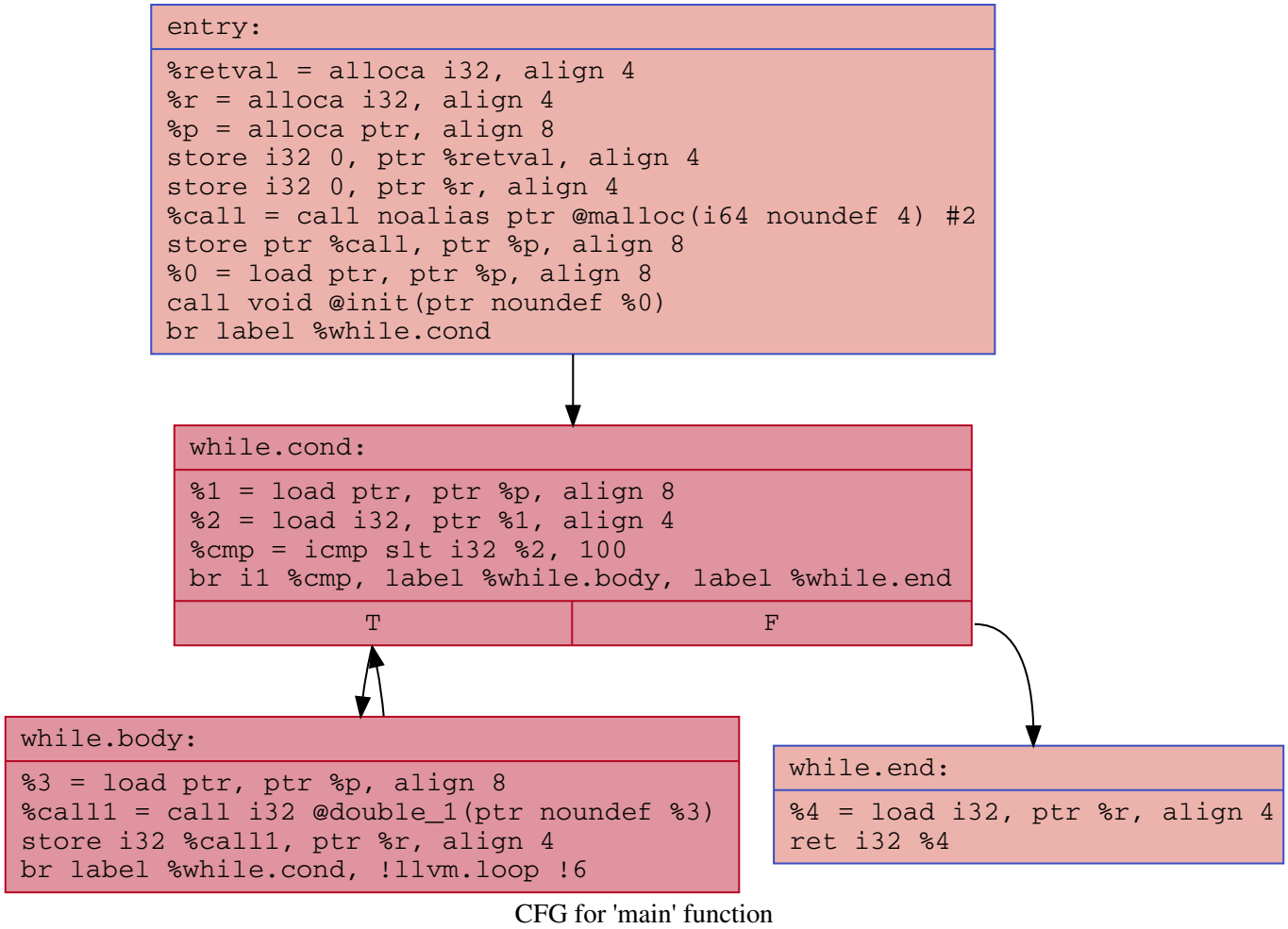


Fig. 2: Example Control-Flow Graph (CFG) using LLVM `-dot-cfg` pass. While technically accurate, the visualization prioritizes information density over visual clarity.

processing in external tools, completely defeating the purpose of automated generation.

In essence, LLVM provides extraordinarily powerful analysis capabilities but pairs them with visualization tools that assume their primary consumer is someone who already understands what they are looking at. This creates a frustrating gap: rich semantic information about program behavior can be automatically extracted, but it cannot be automatically presented in a way that serves educational or presentational contexts. LLVManim was built to bridge this gap by maintaining LLVM's analytical rigor while replacing its austere output with animations designed for human communication rather than machine precision.

C. The Manim Animation Library

The Manim library [7] was originally developed by Grant Sanderson [8] for his educational YouTube channel "3Blue1Brown" [9] to create animations of mathematical and computer-science concepts. Since its release, the channel has accumulated over 6 million subscribers, demonstrating the educational impact of smooth, precisely choreographed animations (Fig. 3) that build intuition about abstract concepts. Given this history, Manim represented an excellent candidate for creating high-quality educational visualizations of program behavior.

Manim's strength lies in its ability to programmatically generate animations with mathematical precision. Instead of tedious keyframe animation, users write Python code to determine how objects should appear, change, and interact. Manim can render 3D scenes, typeset LaTeX equations, and output crisp 60 fps animations suitable for anything from YouTube tutorials to academic conferences. For education, its capacity to make complicated ideas—like matrix transformations, function composition, or program flow—tangible and understandable by systematically unfolding them is particularly valuable.

Yet this power comes with a barrier to entry. A professor preparing a single lecture, or a researcher creating visualizations for a conference talk, may have a clear idea of what they want to show—perhaps how a recursive function builds up its stack—but writing Manim code for it requires a distinct

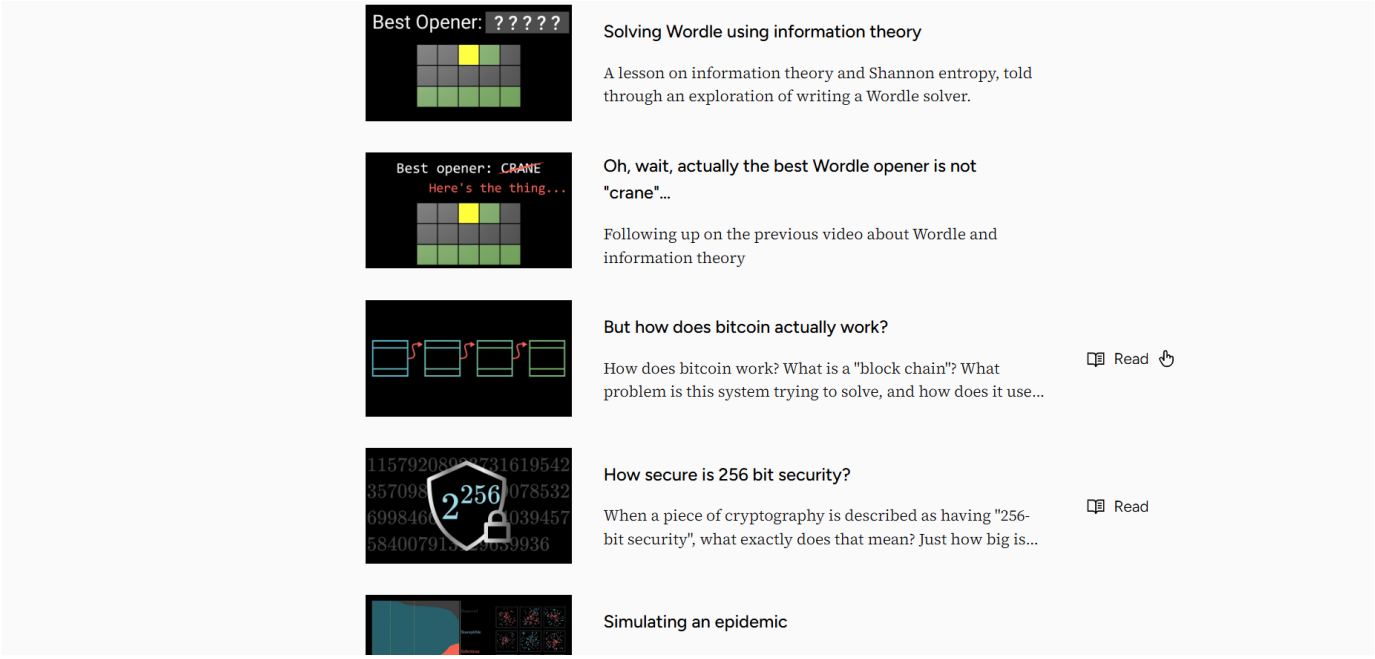


Fig. 3: Examples of 3Blue1Brown videos on Computer Science topics [10]. The thumbnails show a variety of animation styles, all rendered with Manim.

skill set. LLVManim addressed this challenge by automatically translating program semantics into Manim scenes, thereby eliminating the barrier of manual coding between LLVM's analytical power and Manim's visual sophistication.

III. LLVManim System Design

LLVManim bridges the gap between two observations:

1) Methods exist to quickly generate code visualizations, but they produce output that is unappealing for presentation purposes.
2) The Manim library can produce visually impressive animations, but using it requires significant manual effort.

By automating the translation from LLVM IR to Manim scenes, educators and tech leads alike can produce presentation-ready visualizations without needing to compromise between style and accessibility.

A. Design Goals

The following design requirements guided the implementation of LLVManim:

- **Simple CLI:** A command-line interface that accepts `.ll` files, selects visualization options via flags, and produces output with minimal configuration.
- **LLVM IR Parsing:** Extraction of control flow, memory operations, and SSA value information from LLVM IR using the `LLVMLite` [11] Python binding.
- **Multiple Animation Modes:** Several rendering modes offering different levels of detail—from a compact stack-only view to a full three-column layout showing IR source, SSA values, and the call stack simultaneously.
- **IR Instruction Coverage:** Support for the most common LLVM IR instructions (`alloca`, `load`, `store`, `call`, `ret`, `br`, binary operations, and comparisons), enabling visualization of a wide range of programs.
- **Machine-Readable Export:** JSON scene graph and Graphviz DOT output for integration with external tools and downstream analysis.
- **Testability and Quality:** A comprehensive automated test suite and multi-stage quality gate to maintain correctness throughout development.

Deliverables:

- A command-line tool that accepts LLVM IR as input and produces animations, DOT graphs, or JSON scene data as output.
- A library of reusable Manim scene components for visualizing control flow and stack operations.
- Sample visualizations demonstrating LLVManim on standard test programs.
- Documentation covering installation, usage, CLI flags, and supported IR constructs.
- A test suite of 302 automated tests with 84% line coverage.

B. Architecture

LLVManim is implemented as a three-layer pipeline—**Ingestion**, **Transformation**, and **Render**—orchestrated by a dedicated **CLI** entry point. The high-level structure is shown in Fig. 5. Layer boundaries are enforced at build time by `import-linter`, ensuring that no module imports from a downstream layer. The entire codebase is written in Python 3.12 and managed with `uv` [12].

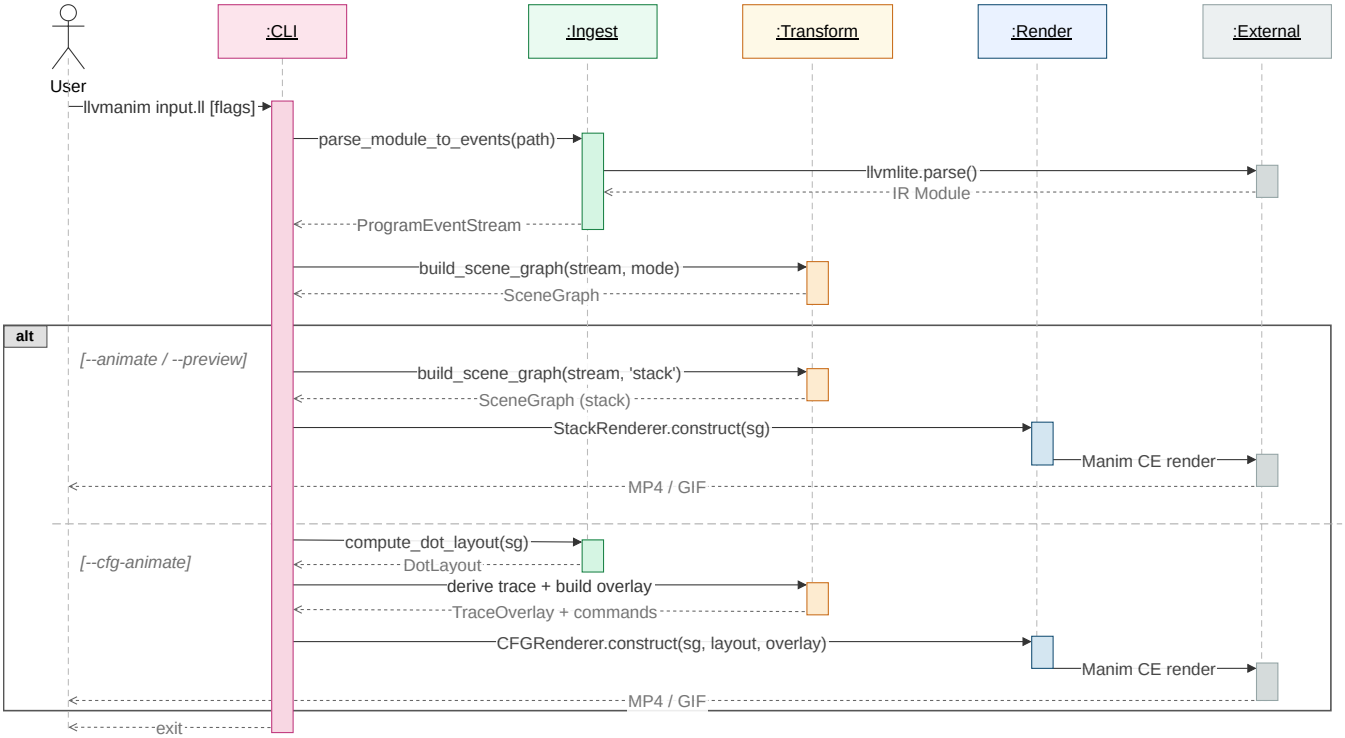


Fig. 4: CLI dispatch flow. Each command flag (`--json`, `--draw`, `--animate`, `--cfg-animate`) shares the Ingest pipeline and branches to the appropriate Transform and Render stages.

1) CLI Entry Point: The `llvmanim.cli` module is the user-facing entry point. It accepts LLVM IR files and orchestrates the three pipeline layers via `argparse`; example usage is shown in CLI 1.

```
# Export JSON scene graph + Graphviz DOT/PNG
uv run llvmanim input.ll --json --draw --outdir
↪  out/
# Render basic stack animation
uv run llvmanim input.ll --animate --ir-mode
↪  basic --outdir out/
# Render rich IR spotlight with SSA panel
uv run llvmanim input.ll --animate --ir-mode
↪  rich-ssa --speed 1.5 --outdir out/
# CFG traversal animation
uv run llvmanim input.ll --cfg-animate --dot-cfg
↪  main.dot --import-trace trace.json --outdir
↪  out/
```

CLI 1: Basic usage

The CLI supports multiple modes of operation, including JSON export, static CFG drawing, stack animation with three IR detail levels, and CFG traversal animation with optional trace import. Each command flag routes through the shared Ingest pipeline to the appropriate Transform and Render stages, as detailed in Fig. 4. When the tool encounters unsupported IR constructs, missing debug metadata, or malformed input, it emits a descriptive error message and continues processing the remainder of the program where possible.

2) Ingestion Layer: The Ingestion layer uses the `LLVMLite` Python binding to parse program structure from LLVM IR in pre-compiled `.ll` files.

- **`llvm_events.py`:** Parses an IR module and emits a `ProgramEventStream`—a typed sequence of `IREvent` records. Each event carries the function name, block name, opcode, full instruction text, its index within the function, debug line number, operand list, and a classified `EventKind`. Table I shows the opcode-to-`EventKind` classification rules. Typed CFG edges are extracted directly from `LLVMLite` terminator operands (`br`, `switch`, `invoke`, `indirectbr`, `callbr`) with true/false labels on conditional branches.

TABLE I: Opcode-to-`EventKind` classification in `llvm_events.py`.

LLVM IR Opcode(s)	EventKind
`alloca`	`alloca`
`load`	`load`
`store`	`store`
`add`, `sub`, `mul`, `div`, etc.	`binop`
`icmp`, `fcmp`	`compare`
`call`	`call`
`ret`	`ret`
`br`	`br`
all others	`other`

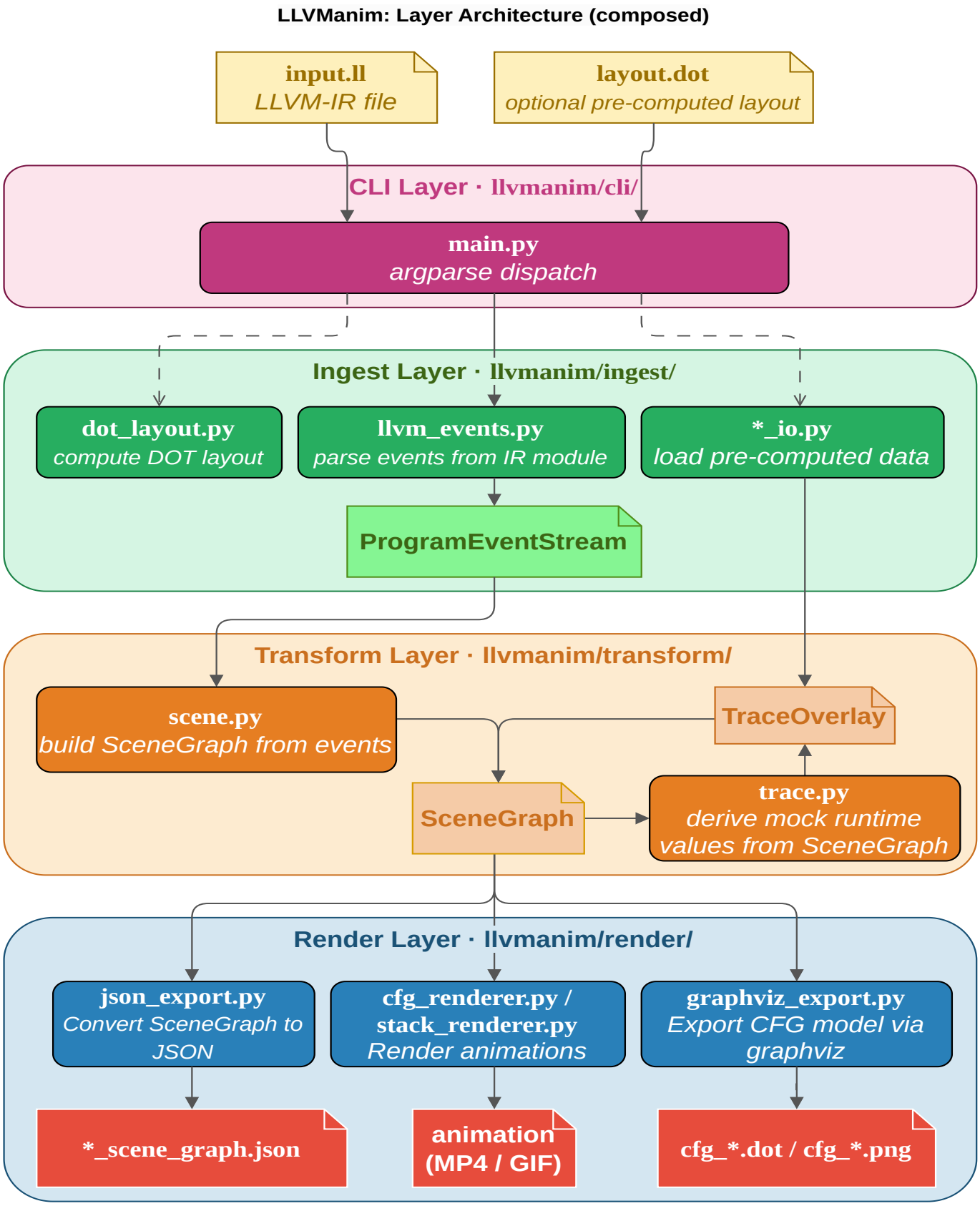


Fig. 5: Detailed module-level architecture. The Ingestion layer collects LLVM IR and metadata, the Transformation layer maps it to an intermediate scene graph, and the Rendering layer renders animations or exports artifacts.

- **cfg_edge_io.py:** Serializes and deserializes CFGEdge lists to JSON, enabling round-trip import/export of control-flow edges via dedicated CLI flags.
- **analysis_metadata_io.py:** Loads and saves dominator-tree and loop-analysis metadata as JSON, supporting round-trip import and export via CLI flags.
- **trace_io.py:** Reads and writes runtime path traces as JSON (--import-trace / --export-trace), used for overlay visualization in CFG traversal animations.
- **dot_layout.py:** Parses a Graphviz DOT file produced by opt -passes=dot-cfg and computes a DotLayout with node positions and edge routes for the CFG animation scene.
- **display_lines.py:** Cleans raw IR text into per-function line lists used by the rich animation panels.

3) Transformation Layer: The Transformation layer converts the ingest output into an intermediate scene graph and execution traces suitable for rendering. The complete data model spanning all three layers is shown in Fig. 6.

- **models.py:** Defines the core data model—SceneGraph, SceneNode, SceneEdge, CFGBlock, TraceOverlay, and BlockMetadata. Also defines AnimationCommand and ActionKind, the typed animation primitives consumed by the Render layer. Nodes carry a kind string and an open properties dict, with an optional animation_hint string. SceneGraph embeds an ordered commands list alongside its nodes and edges.
- **scene.py:** The build_scene_graph function groups events by function and block, attaches typed CFG edges, assigns block roles, applies optional domtree/loop metadata, and populates SceneGraph.commands via the overlay builder. build_stack_scene_graph produces a call-tree-ordered SceneGraph with a fully populated commands list, translating the parsed ProgramEventStream directly into typed AnimationCommand records (carrying ActionKind, target, and params) ready for consumption by the Render layer.
- **trace.py:** The build_execution_trace function produces a list of TraceStep or RichTraceStep records capturing the execution order with optional SSA result values for binop, compare, and load instructions.

4) Render Layer: The Render layer renders the scene graph using Manim Community Edition [13] and provides export functionality. All scenes extend CommandDrivenScene, a base class that iterates over the SceneGraph.commands list and dispatches each AnimationCommand to a registered handler. The complete type hierarchy—from Ingest types through Transform intermediates to the Render dispatch classes—is shown in Fig. 6.

- **Animation Modes:** The tool supports three stack-rendering modes using --ir-mode flags, all driven by StackRenderer—a CommandDrivenScene subclass. Each mode consumes the same AnimationCommand records (detailed in Fig. 6) but presents progressively more detail. Specific information on the implemented modes is provided in the *Results* Section V-C.
- **CFG Traversal:** --cfg-animate renders an animated walk through the control-flow graph using CFGRenderer (another CommandDrivenScene subclass) with DOT-derived node positions (--dot-cfg) and a runtime trace (--import-trace). If no trace is provided, a static DFS trace is derived automatically.
- **Export:** Static artifacts complement the animated output.
  - --json: Writes a machine-readable scene graph (scene_graph.json).
  - --draw: Writes cfg_main.dot and, when Graphviz is available, a rendered cfg_main.png.
- **Rendering:** Manim CE renders MP4 or GIF output. GIF conversion uses an ffmpeg palette workflow to reduce peak memory usage. The --preview flag opens the rendered animation in a viewer.

IV. Risk Evaluation and Mitigation

Prior to implementation, we identified and categorized the primary risks to the project across two dimensions: technical risks arising from the complexity of the target technologies, and architectural risks stemming from design and scope decisions.

A. Technical Risks

- **LLVM IR Complexity:** LLVM IR supports a wide range of constructs, including vector operations, intrinsics, and optimization-introduced transformations. LLVManim targets a common subset of instructions for the initial implementation.
  - *Risk:* Unsupported constructs may cause parsing failures or incomplete visualizations.
  - *Mitigation:* Restrict the MVP to unoptimized -O0 IR and provide clear diagnostic messages when encountering unsupported instructions.
- **Manim Performance:** Manim provides high-quality animations but can be computationally expensive, especially for scenes with many objects or transitions.
  - *Risk:* Long render times may slow development and limit iterative testing.
  - *Mitigation:* Use low-quality preview rendering during development and reserve high-quality output for final deliverables.

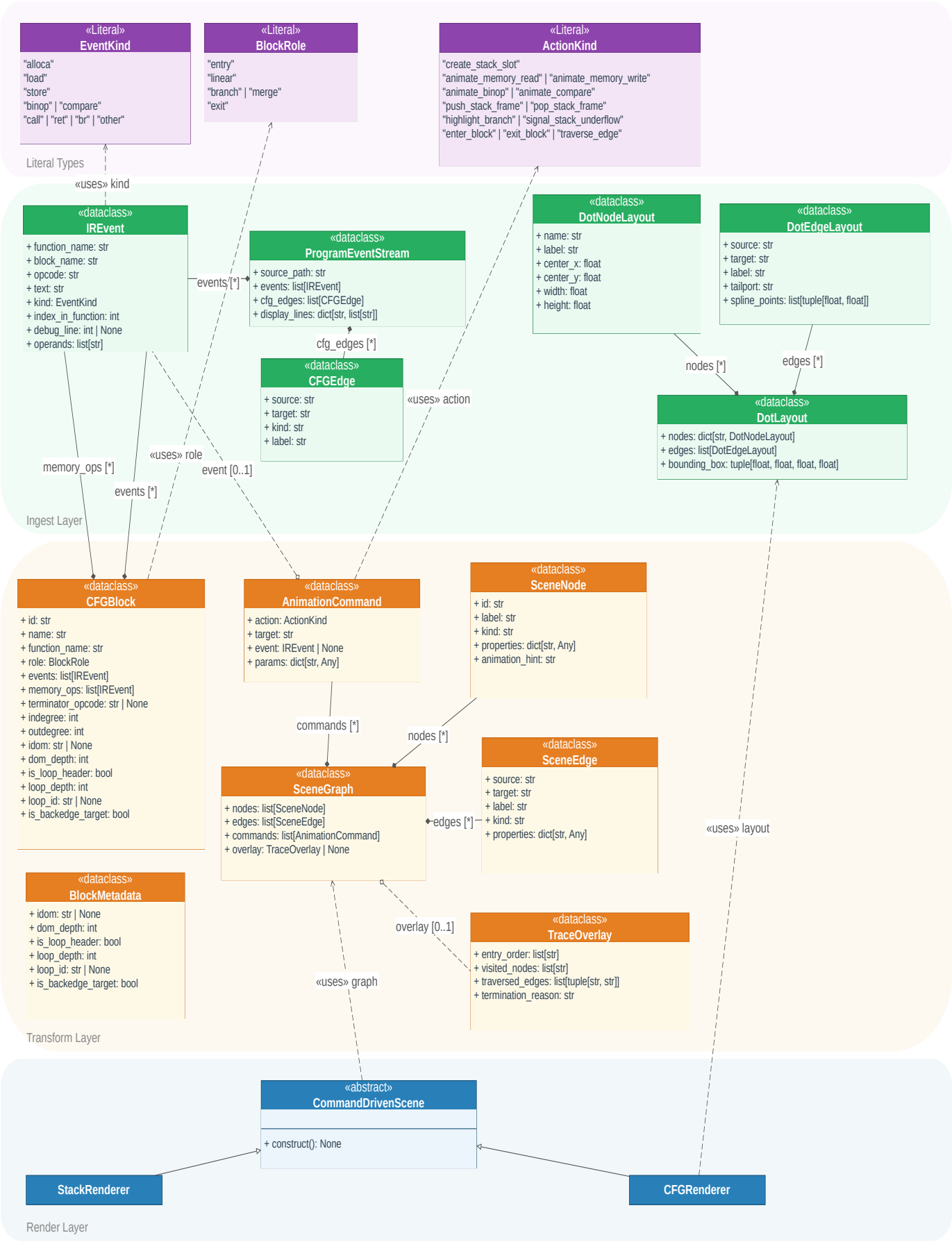


Fig. 6: Unified data model. Ingest types (`ProgramEventStream`, `IREvent`, `CFGEdge`, `DotLayout`) feed into Transform types (`SceneGraph`, `SceneNode`, `SceneEdge`, `AnimationCommand`, `TraceOverlay`) which are consumed by Render types (`CommandDrivenScene`, `StackRenderer`, `CFGRenderer`).

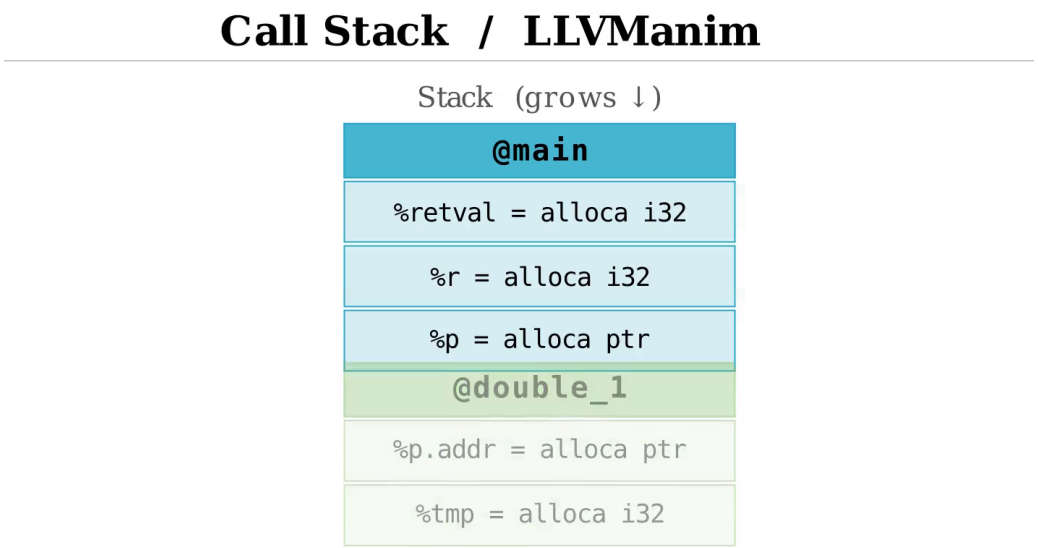


Fig. 7: Basic mode: stack-only view with per-cell badge flash during a `call` to `@init`.

- **Cross-Platform Compatibility:** Achieving consistent LLVManim behavior across operating systems and development environments can be challenging.
  - *Risk:* Platform-specific issues may arise and cause inconsistent behavior or failures.
  - *Mitigation:* Use cross-platform libraries and conduct testing on multiple platforms to ensure compatibility.
- **Dependency Fragility:** LLVManim depends on Clang, LLVMlite, and Manim, each of which may introduce version incompatibilities or platform-specific issues.
  - *Risk:* Changes in LLVM or Manim APIs could break LLVManim's functionality.
  - *Mitigation:* Provide a reproducible environment via containerization or a standardized setup script.

B. Architectural Risks

- **User Expectations for Customization:** Educators and presenters often need fine-grained control over pacing, color schemes, and layout choices.
  - *Risk:* Limited customization may reduce adoption and may fail to meet expectations for presentation-quality output.
  - *Mitigation:* Provide a minimal but meaningful set of customization flags (e.g., palette, speed, verbosity) in the initial release.

V. Results

This section reports the implemented capabilities of LLVManim across its final release, covering feature completeness, the supported subset of LLVM IR instructions, available animation modes, test suite quality, and overall codebase size. Performance metrics and reproducibility details are also included to demonstrate the tool's practical usability and reliability.

A. Feature Coverage

Table II summarizes the proposed features from the original project goals alongside their implementation status. Of the originally proposed proof-of-concept targets, all were achieved except a GUI. Interactive stepping/pause was deemed redundant given the ability to manually control `mp4` playback in most modern media players.

TABLE II: Proposed vs. Implemented Features

Feature	Status
CLI accepting `.ll` files	✓
LLVM IR parsing via `LLVMLite`	✓
Typed CFG edge extraction	✓
Intermediate scene graph	✓
JSON scene graph export	✓
Graphviz DOT/PNG export	✓
Basic stack animation	✓
Rich IR spotlight animation	✓
Rich-SSA 3-column animation	✓
CFG traversal animation	✓
Runtime trace import/export	✓
Domtree/loop metadata import	✓
GIF output with palette workflow	✓
Adjustable animation speed	✓
Automatic C compilation	✓
YAML configuration files	✓
Color palette selection	✓
Interactive stepping/pause	∼
GUI	×

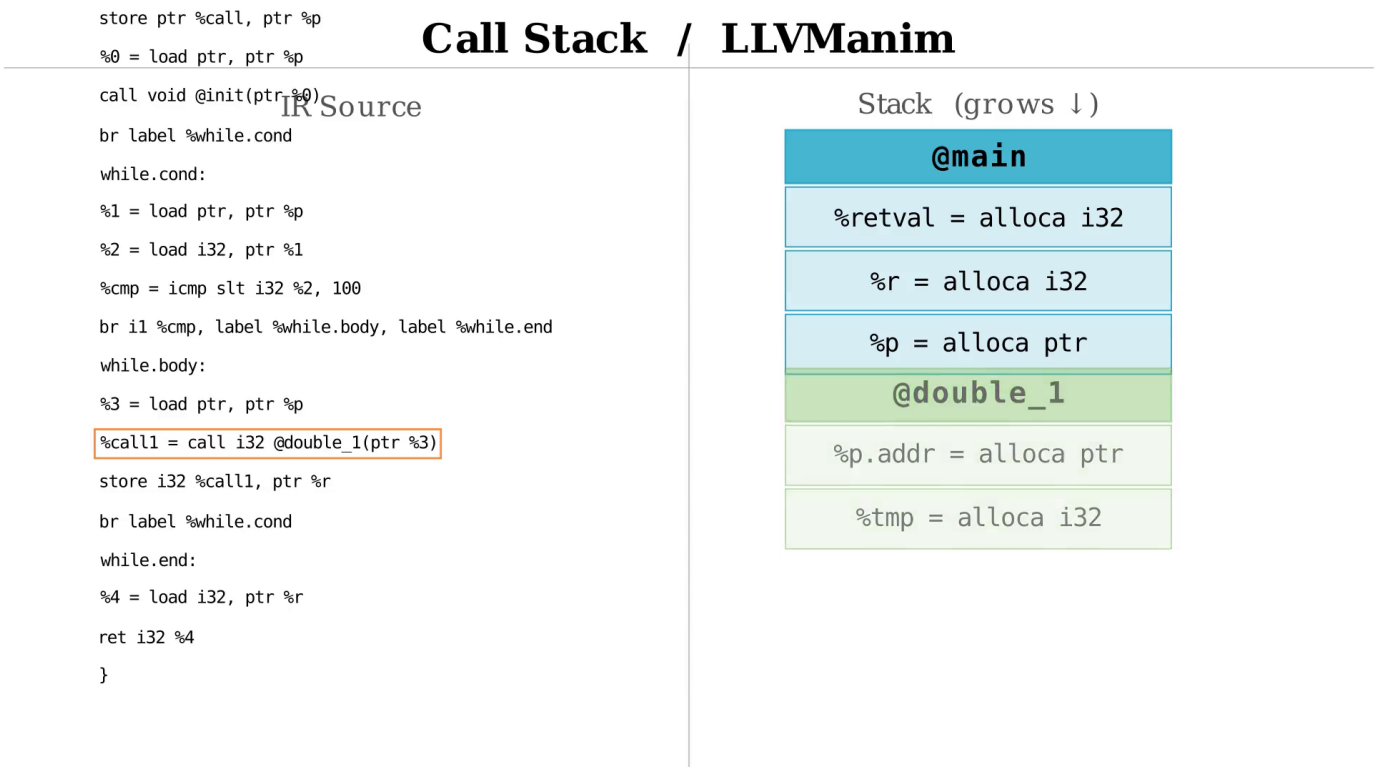


Fig. 8: Rich mode: two-column layout with IR source spotlight alongside the stack.

B. Supported IR Instructions

LLVManim classifies each LLVM IR instruction into one of nine `EventKind` categories:

- `alloca`: Stack memory allocation
- `load` / `store`: Memory read and write operations
- `binop`: Binary arithmetic operations (`add`, `sub`, `mul`, etc.)
- `compare`: Integer and floating-point comparisons (`icmp`, `fcmp`)
- `call`: Function invocations
- `ret`: Function returns
- `br`: Branch instructions (conditional and unconditional)
- `other`: Unsupported or unrecognized instructions (skipped with a diagnostic message)

CFG edge extraction supports five terminator types: `br`, `switch`, `invoke`, `indirectbr`, and `callbr`. Conditional branches are labeled with T/F direction annotations.

C. Animation Modes

LLVManim provides three stack-focused animation modes and one graph-based mode, each reproducible with a single CLI command. The stack-focused modes (basic, rich, rich-SSA) share a common layout and animation style, with increasing levels of IR detail. The graph-based mode renders an animated traversal of the control-flow graph using Graphviz layout and a runtime trace for the traversal order. Each mode is designed to target different presentation contexts and audience familiarity levels, allowing users to choose the appropriate level of detail for their needs. The following sections describe each mode in detail, along with example CLI commands for generating the animations. All animations were generated from the same `double.ll` test program, which contains a simple loop with arithmetic operations, memory accesses, and control flow. The full source code for `double.c` and `double.ll` is provided in appendix A.

Basic mode (`--ir-mode basic`) renders a minimal stack-only view. Each IR instruction triggers a per-cell badge flash on the corresponding stack slot, providing a clean visual pulse of execution without additional clutter. This mode is best suited for audiences who need only a high-level sense of execution flow, such as introductory lectures or quick demonstrations, since it strips away all source-level detail and focuses purely on the call stack. CLI 2 shows the CLI command used to generate the animation shown in Fig. 7.

```
uv run llvmanim double.ll --animate --ir-mode
↪  basic --outdir out/
```

CLI 2: Command to generate the basic mode animation shown in Fig. 7.

Rich mode (`--ir-mode rich`) extends the basic view with a two-column layout. An IR source panel on the left displays the current function's instructions while a spotlight cursor tracks the executing line, giving the audience both the "what" (the stack) and the "where" (the source). This mode is particularly valuable when the presenter wants to walk through specific IR constructs—for example, showing how a series

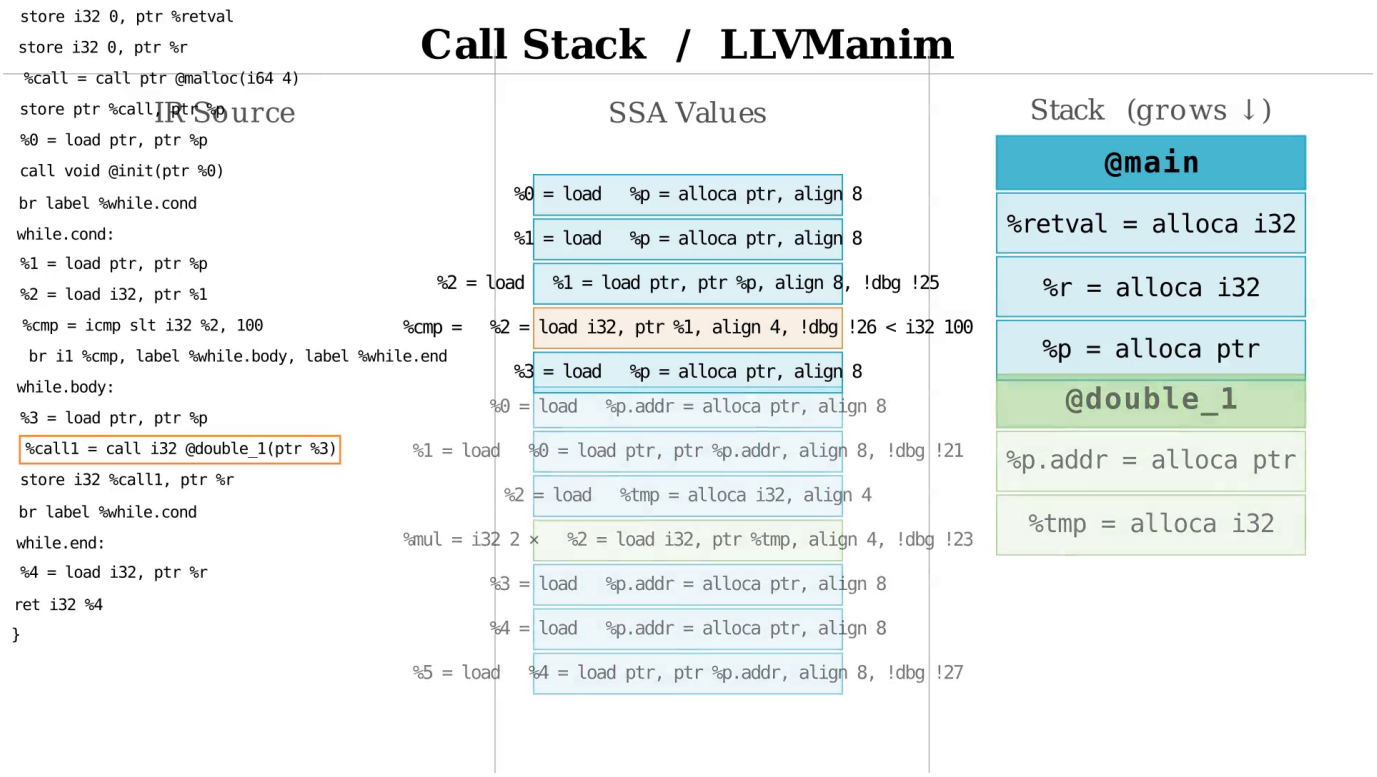


Fig. 9: Rich-SSA mode: three-column layout showing computed SSA values for `load` and `icmp` instructions.

of `alloca` and `store` instructions initialize local variables before a loop body executes. The spotlight cursor provides a natural focal point that guides the viewer's attention through the code. CLI 3 shows the CLI command used to generate the animation shown in Fig. 8.

```
uv run llvmanim double.ll --animate --ir-mode
↪  rich --outdir out/
```

CLI 3: Command to generate a rich mode animation shown in Fig. 8.

Rich-SSA mode (`--ir-mode rich-ssa`) takes the rich layout a step further with a dedicated third column for SSA values (IR Source | SSA Values | Stack). The center panel displays the symbolic result of each binary operation, comparison, or load as it executes, making it possible to follow data dependencies without mentally resolving `%`-names. This is the most detailed mode and targets audiences already familiar with SSA form—for instance, a compiler course where students need to see how `%4 = icmp slt i32 %2, %3` evaluates during a loop iteration. CLI 4 shows the CLI command used to generate the animation shown in Fig. 9.

```
uv run llvmanim double.ll --animate --ir-mode
↪  rich-ssa --outdir out/
```

CLI 4: Command to generate a rich-SSA mode animation shown in Fig. 9.

CFG traversal (`--cfg-animate`) renders an animated walk through the control-flow graph, using Graphviz DOT layout for node positions and a runtime trace for the traversal order:

```
uv run llvmanim double.ll --cfg-animate --dot-cfg
↪  main.dot --import-trace trace.json --outdir
↪  out/
```

CLI 5: Command to generate a CFG traversal animation.

D. Test Coverage and Quality

The LLVManim implementation is validated by 302 automated tests organized into four marker categories:

TABLE III: Test Distribution by Category

Category	Count
Unit (ingest, transform)	192
Integration (present, cli, pipeline)	96
Contract (export boundaries)	8
End-to-end (entrypoints)	6
Total	**302**

Line coverage stands at 84% across the entire `llvmanim` package. Fig. 11a (Appendix B) shows per-module coverage and Fig. 11b shows the test distribution across modules. A four-stage quality gate runs on every pull request:

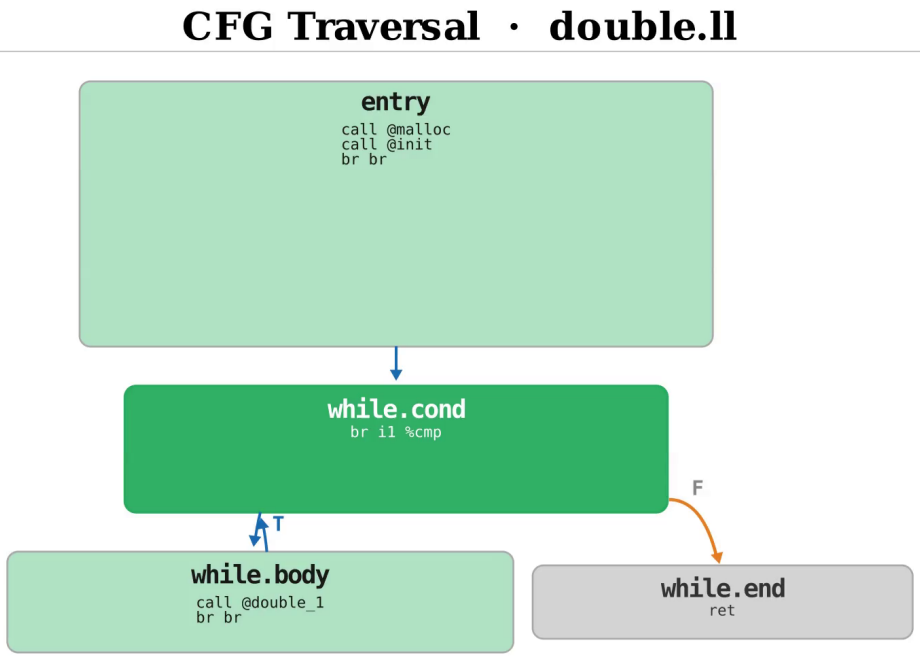


Fig. 10: CFG mode: Animated walk through the control-flow graph, highlighting blocks and edges as they are traversed.

1) **Ruff**: Linting and import sorting
2) **Pyright**: Static type checking (strict mode)
3) **import-linter**: Architectural boundary enforcement between layers
4) **pytest**: Full test suite execution with coverage reporting

E. Codebase Size

The LLVManim codebase comprises 3,189 lines of Python across 18 modules. Fig. 12 (Appendix B) shows the breakdown by layer and module. The Render layer accounts for 42% of the code, followed by Ingest (25%), Transform (19%), and CLI (14%).

F. Performance

All pipeline stages exhibit linear scaling behavior, confirming that LLVManim can handle realistically sized LLVM IR inputs. Benchmarks use synthetic IR programs with controlled instruction and block counts (median of 5 runs per data point).

Fig. 13 (Appendix B) shows the relative wall-clock cost of each pipeline stage for a 50-block CFG. The Ingest stage (`parse_module_to_events`) dominates at 11.6 ms, while all Transform stages complete in under 0.25 ms. Fig. 14 confirms that parse time scales linearly with instruction count—$\Theta(n)$ where n is the number of IR instructions.

The scene-graph construction stages also scale linearly: `build_scene_graph(mode='cfg')` is $\Theta(n_b)$ in block count (Fig. 15), `derive_cfg_trace` is $\Theta(n_b)$ for linear-chain CFGs and $O(n_b \cdot k)$ in the presence of loops (Fig. 16), and `build_scene_graph(mode='stack')` exhibits sub-linear growth empirically (Fig. 17).

Peak memory allocation also grows linearly with IR size for both pipelines (Fig. 18).

G. Reproducibility

All LLVManim results can be reproduced from the public repository [14]:

```
git clone
↪  https://github.com/BenHoule/LLVManim.git
cd LLVManim
uv sync --dev
# run all 4 quality gates
./scripts/quality-check.sh
# Create coverage and LoC figures.
./uv run python docs/metrics/generate_metrics.py
# Create pipeline latency and scaling figures.
./uv run python
↪  docs/metrics/generate_perf_metrics.py
```

CLI 6: Commands to reproduce all results.

System requirements: Python 3.12+, `uv`, Linux packages `libcairo2` and `libpango` (for Manim), and optionally `graphviz` for PNG graph export. See the repository's `README.md` for detailed setup instructions and troubleshooting tips.

VI. Discussion

This section reflects on the development process, examining the key architectural decisions made during implementation, where the final system diverged from the original proposal, the known limitations that remain unresolved, and lessons learned over the course of the project.

A. Architectural Decisions

Several architectural decisions shaped the final system—

- **Three-layer separation with import-linter:** Enforcing strict layer boundaries at build time prevented coupling between Ingestion, Transformation, and Rendering. This paid dividends during the IR display model refactor, where the Transformation layer's data structures were restructured without touching any Rendering layer code. The `import-linter` tool caught accidental cross-layer imports on multiple occasions during development.
- **Display model abstraction:** Rather than having Manim scenes query the scene graph directly, we introduced an explicit display model (the `TraceStep`/`RichTraceStep` and `AnimationCommand` types) that decouples rendering from data extraction. This made it straightforward to add the rich-SSA animation mode by extending the trace step type without modifying existing scene classes.
- **Pipeline approach:** The tool processes data in a linear pipeline: parse → build scene graph → render or export. This simplified testing because each stage could be exercised independently with deterministic fixtures. The 302 tests in the suite are predominantly unit tests that operate on a single layer.
- **LLVMLite for terminator parsing:** Early prototypes used regex-based parsing of LLVM IR text, which proved fragile for edge cases such as `switch` and `invoke` terminators. Switching to `LLVMLite`'s Python binding, which exposes the LLVM C API, made CFG edge extraction reliable across all five terminator types.

B. Deviations from Proposal

The following proposed features were not implemented—

- **Interactive stepping and pause:** Interactive playback was deprioritized in favor of richer static animation modes. Manim CE's rendering model produces video files rather than interactive sessions, making pause/step less natural.
- **YAML configuration and –palette:** Fine-grained customization via YAML files and named color palettes was deferred. The current CLI provides `--speed` and `--format` as the primary customization points.
- **GUI:** A graphical interface was listed as a stretch goal and was not pursued given the limited timeline.

C. Known Limitations

The current implementation has several known limitations—

- **Double-pop bug:** Under certain control-flow patterns involving consecutive returns, the basic animation mode may attempt to pop the stack twice for a single `ret` instruction. This is tracked as a known issue.
- **Symbolic-only SSA values:** The rich-SSA panel displays symbolic operand names (e.g., `%3 = add %1, %2`) rather than concrete runtime values. A single swap-point in `ssa_formatting.py` exists for when numeric runtime values are plumbed through in a future version.
- **No heap visualization:** The tool visualizes stack operations but does not yet render heap allocations or pointer-based data structures.
- **Manim render times:** Full-quality Manim renders can take significant time for programs with many instructions. The `--speed` flag and low-quality preview mode help during development, but render time remains a practical constraint for large programs.

D. Lessons Learned

- **TDD was essential for refactors:** The test suite caught regressions immediately when refactoring the IR display model and restructuring the scene graph. Without comprehensive tests, these refactors would have been significantly riskier.
- **Architectural enforcement catches violations early:** `import-linter` rejected several pull requests that would have introduced cross-layer dependencies, preventing technical debt before it accumulated.
- **Sandbox-driven exploration:** Experimental Manim scripts in the `sandbox/` directory allowed rapid prototyping of animation ideas (CFG traversal, register panels) without risking the main codebase.
- **Typed CFG edges via LLVMLite:** Moving from regex-based IR text parsing to `LLVMLite`'s terminator API eliminated an entire class of parsing bugs and made the Ingestion layer robust against IR formatting variations.

VII. Conclusion

We presented LLVManim, a tool that automatically generates presentation-quality animations from LLVM IR. The system's three-layer pipeline—Ingestion, Transformation, and Rendering—cleanly separates concerns and is enforced by architectural boundary checks at build time. Over a single semester, we delivered a fully functional CLI tool with three animation modes, a CFG traversal visualizer, machine-readable export formats, and a comprehensive test suite of 302 tests at 84% line coverage.

LLVManim demonstrates that the gap between LLVM's powerful but utilitarian analysis output and the polished animations achievable with Manim can be bridged through thoughtful automation. By targeting LLVM IR as input, the tool inherits support for any language that compiles through the LLVM toolchain, making it broadly applicable to educational and presentational contexts.

Future Work

Several directions remain for future development:

- **Numeric SSA values:** Plumbing concrete runtime values through the trace overlay to replace symbolic operand names in the rich-SSA panel.
- **Heap visualization:** Extending the animation modes to render heap allocations, pointer dereferences, and linked data structures.

- **Double-pop fix:** Resolving the known bug in consecutive-return control-flow patterns.
- **Interactive playback:** Exploring Manim's OpenGL preview mode or alternative rendering backends for pause/step interactivity.

ACKNOWLEDGMENT

We would like to thank Dr. James Daly for his guidance, feedback, and support throughout the development of this work.

REFERENCES

[1] P. J. Guo, "PythonTutor: Visualize code execution," https://pythontutor.com, 2010, accessed: Feb. 13, 2026.

[2] ——, "Online python tutor: embeddable web-based program visualization for cs education," in *Proceeding of the 44th ACM Technical Symposium on Computer Science Education*, ser. SIGCSE '13. New York, NY, USA: Association for Computing Machinery, 2013, pp. 579–584. [Online]. Available: https://doi.org/10.1145/2445196.2445368

[3] ——, "Codeopticon: Real-time, one-to-many human tutoring for computer programming," in *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*, ser. UIST '15. New York, NY, USA: Association for Computing Machinery, 2015, pp. 599–608. [Online]. Available: https://doi.org/10.1145/2807442.2807469

[4] C. Lattner and V. Adve, "LLVM: A compilation strategy for lifelong program analysis and transformation," in *Proceedings of the 2nd IEEE Annual CGO and ACM SIGPLAN Annual CC Symposium*, ser. CGO-CC '04. Palo Alto, CA, USA: IEEE Computer Society, 2004, pp. 60–72.

[5] LLVM Project, "LLVM language reference manual," https://llvm.org/docs/LangRef.html, 2024, accessed: 2026-02-25.

[6] J. Ellson, E. Gansner, E. Koutsofios, S. North, and G. Woodhull, "Graphviz – Open Graph Visualization Software," https://graphviz.org/, 2001, accessed: Apr. 17, 2026.

[7] G. Sanderson, "Manim – mathematical animation framework," https://github.com/3b1b/manim, 2015, accessed: Feb. 13, 2026.

[8] ——, "3blue1brown," Website, 2026, accessed: Feb. 13, 2026. [Online]. Available: https://www.3blue1brown.com/

[9] ——, "3blue1brown," Youtube Channel, March 2015, accessed: Apr. 27, 2026. [Online]. Available: https://www.youtube.com/@3blue1brown

[10] 3Blue1Brown, "Topics: Computer science," https://www.3blue1brown.com/?topic=computer-science, 2023, accessed: Apr. 27, 2026.

[11] Numba Development Team, "llvmlite: A lightweight llvm python binding for writing jit compilers," https://github.com/numba/llvmlite, 2024, accessed: 2026-02-25.

[12] Astral Software, *uv: An Extremely Fast Python Package and Project Manager*, Astral, 2026, accessed: 2026-04-21. [Online]. Available: https://astral.sh/uv

[13] Manim Community Developers, "Manim community edition," https://www.manim.community/, 2024, accessed: Apr. 17, 2026.

[14] B. Houle, E. Oloyede, L. Grippo, and M. Ahmed, "LLVManim: Presentable debugging with LLVM and Manim," https://github.com/BenHoule/LLVManim, 2026, accessed: Apr. 17, 2026.

Appendix A
Input Files: double.c and double.ll

double.c is a minimal C program used to generate the example LLVManim output. It defines three functions—init, double_1, and main—whose control flow, heap allocation, and loop structure exercise all four LLVManim animation modes.

```c
#include <stdlib.h>

void init(int *p) {
  *p=1;
}

int double_1(int *p){
  int tmp = *p;
  *p = 2*tmp;
  return (*p);
}

int main() {
  int r = 0;
  int *p = (int *) malloc(sizeof(int));
  init(p);
  while (*p < 100) {
    r = double_1(p);
  }
  return r;
}
```

Source Code 1: Full C source (double.c) used for all sample animations.

double.ll is compiled at -O0 with full debug info (clang -O0 -g -S -emit-llvm double.c). It is the actual LLVM IR input used by LLVManim in the examples.

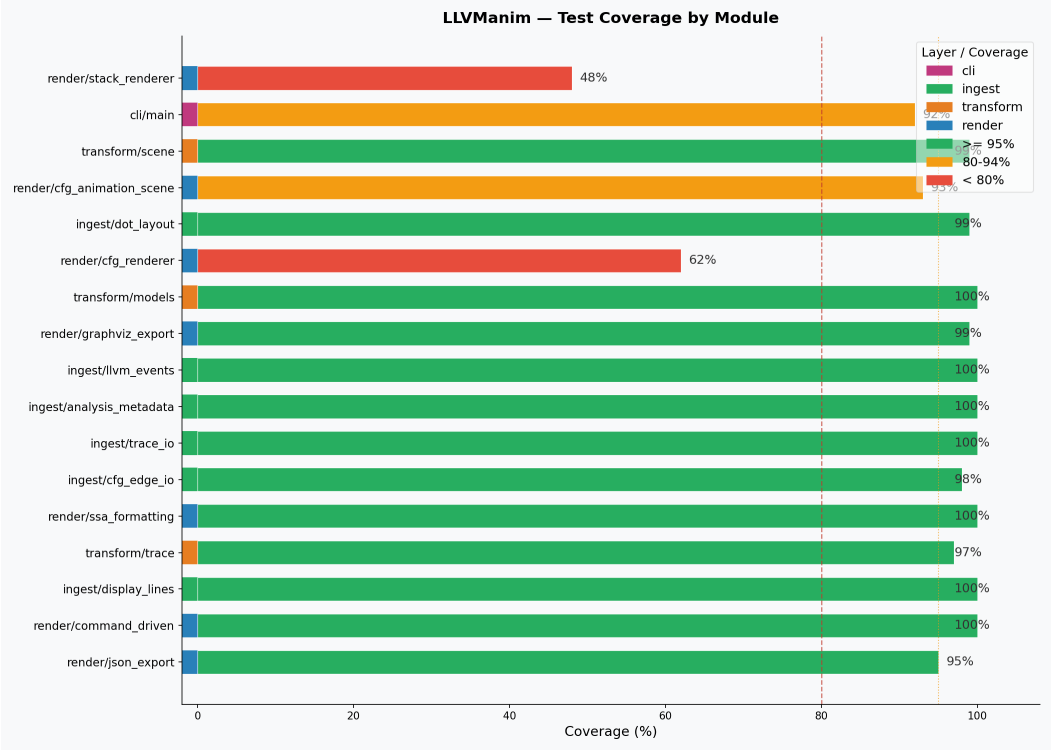
```llvm
; ModuleID = 'double.c'
source_filename = "double.c"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-f80:128-n8:16:32:64-S128"
target triple = "x86_64-pc-linux-gnu"

; Function Attrs: noinline nounwind optnone uwtable
define dso_local void @init(ptr noundef %p) #0 !dbg !13 {
entry:
  %p.addr = alloca ptr, align 8
  store ptr %p, ptr %p.addr, align 8
  call void @llvm.dbg.declare(metadata ptr %p.addr, metadata !17, metadata !DIExpression()), !dbg !18
  %0 = load ptr, ptr %p.addr, align 8, !dbg !19
  store i32 1, ptr %0, align 4, !dbg !20
  ret void, !dbg !21
}

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn memory(none)
declare void @llvm.dbg.declare(metadata, metadata, metadata) #1

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @double_1(ptr noundef %p) #0 !dbg !22 {
entry:
  %p.addr = alloca ptr, align 8
  %tmp = alloca i32, align 4
  store ptr %p, ptr %p.addr, align 8
  call void @llvm.dbg.declare(metadata ptr %p.addr, metadata !25, metadata !DIExpression()), !dbg !26
  call void @llvm.dbg.declare(metadata ptr %tmp, metadata !27, metadata !DIExpression()), !dbg !28
  %0 = load ptr, ptr %p.addr, align 8, !dbg !29
  %1 = load i32, ptr %0, align 4, !dbg !30
  store i32 %1, ptr %tmp, align 4, !dbg !28
  %2 = load i32, ptr %tmp, align 4, !dbg !31
  %mul = mul nsw i32 2, %2, !dbg !32
```

```
  %3 = load ptr, ptr %p.addr, align 8, !dbg !33
  store i32 %mul, ptr %3, align 4, !dbg !34
  %4 = load ptr, ptr %p.addr, align 8, !dbg !35
  %5 = load i32, ptr %4, align 4, !dbg !36
  ret i32 %5, !dbg !37
}

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 !dbg !38 {
entry:
  %retval = alloca i32, align 4
  %r = alloca i32, align 4
  %p = alloca ptr, align 8
  store i32 0, ptr %retval, align 4
  call void @llvm.dbg.declare(metadata ptr %r, metadata !41, metadata !DIExpression()), !dbg !42
  store i32 0, ptr %r, align 4, !dbg !42
  call void @llvm.dbg.declare(metadata ptr %p, metadata !43, metadata !DIExpression()), !dbg !44
  %call = call noalias ptr @malloc(i64 noundef 4) #3, !dbg !45
  store ptr %call, ptr %p, align 8, !dbg !44
  %0 = load ptr, ptr %p, align 8, !dbg !46
  call void @init(ptr noundef %0), !dbg !47
  br label %while.cond, !dbg !48

while.cond:                                       ; preds = %while.body, %entry
  %1 = load ptr, ptr %p, align 8, !dbg !49
  %2 = load i32, ptr %1, align 4, !dbg !50
  %cmp = icmp slt i32 %2, 100, !dbg !51
  br i1 %cmp, label %while.body, label %while.end, !dbg !48

while.body:                                       ; preds = %while.cond
  %3 = load ptr, ptr %p, align 8, !dbg !52
  %call1 = call i32 @double_1(ptr noundef %3), !dbg !54
  store i32 %call1, ptr %r, align 4, !dbg !55
  br label %while.cond, !dbg !48, !llvm.loop !56

while.end:                                        ; preds = %while.cond
  %4 = load i32, ptr %r, align 4, !dbg !59
  ret i32 %4, !dbg !60
}

; Function Attrs: nounwind allocsize(0)
declare noalias ptr @malloc(i64 noundef) #2

attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #1 = { nocallback nofree nosync nounwind speculatable willreturn memory(none) }
attributes #2 = { nounwind allocsize(0) "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #3 = { nounwind allocsize(0) }

!llvm.dbg.cu = !{!0}
!llvm.module.flags = !{!5, !6, !7, !8, !9, !10, !11}
!llvm.ident = !{!12}

!0 = distinct !DICompileUnit(language: DW_LANG_C11, file: !1, producer: "Ubuntu clang version 18.1.3 (1ubuntu1)", isOptimized: false, runtimeVersion: 0, emissionKind: FullDebug, retainedTypes: !2, splitDebugInlining: false, nameTableKind: None)
!1 = !DIFile(filename: "double.c", directory: "/home/benh/LLVManim", checksumkind: CSK_MD5, checksum: "0f941b637837176c30f52c4a0313c1b1")
!2 = !{!3}
!3 = !DIDerivedType(tag: DW_TAG_pointer_type, baseType: !4, size: 64)
!4 = !DIBasicType(name: "int", size: 32, encoding: DW_ATE_signed)
!5 = !{i32 7, !"Dwarf Version", i32 5}
!6 = !{i32 2, !"Debug Info Version", i32 3}
!7 = !{i32 1, !"wchar_size", i32 4}
!8 = !{i32 8, !"PIC Level", i32 2}
!9 = !{i32 7, !"PIE Level", i32 2}
!10 = !{i32 7, !"uwtable", i32 2}
!11 = !{i32 7, !"frame-pointer", i32 2}
!12 = !{!"Ubuntu clang version 18.1.3 (1ubuntu1)"}
```

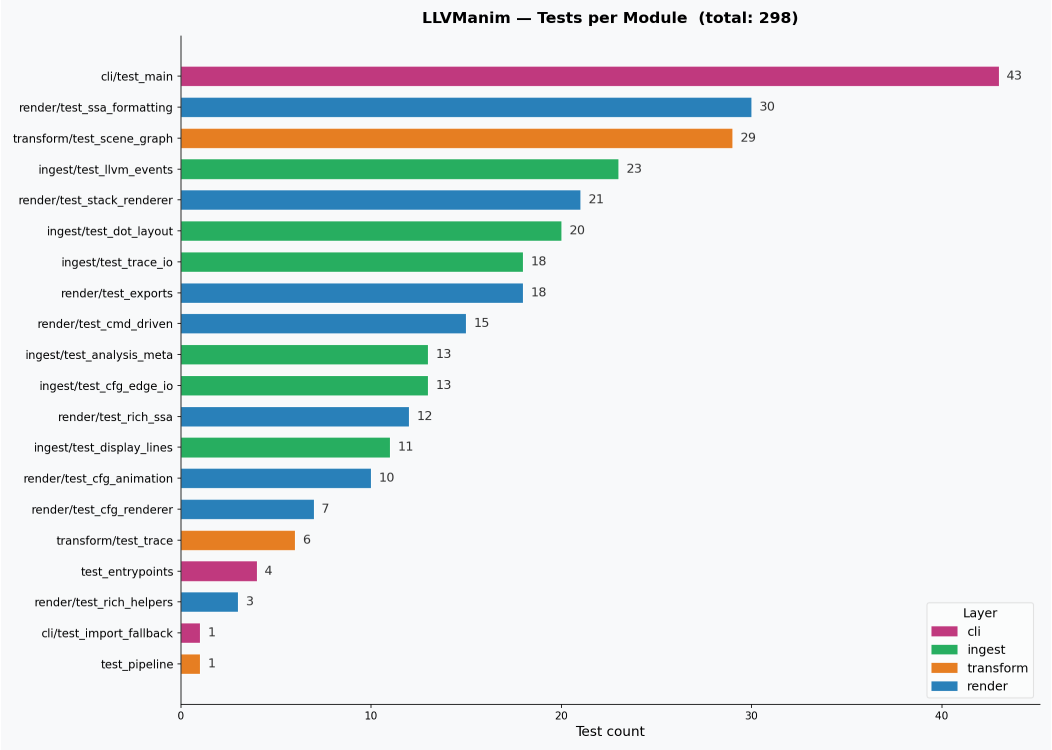
```
!13 = distinct !DISubprogram(name: "init", scope: !1, file: !1, line: 5, type: !14, scopeLine: 5, flags: DIFlagPrototyped, spFlags: DISPFlagDefinition, unit: !0, retainedNodes: !16)
!14 = !DISubroutineType(types: !15)
!15 = !{null, !3}
!16 = !{}
!17 = !DILocalVariable(name: "p", arg: 1, scope: !13, file: !1, line: 5, type: !3)
!18 = !DILocation(line: 5, column: 16, scope: !13)
!19 = !DILocation(line: 6, column: 4, scope: !13)
!20 = !DILocation(line: 6, column: 5, scope: !13)
!21 = !DILocation(line: 7, column: 1, scope: !13)
!22 = distinct !DISubprogram(name: "double_1", scope: !1, file: !1, line: 9, type: !23, scopeLine: 9, flags: DIFlagPrototyped, spFlags: DISPFlagDefinition, unit: !0, retainedNodes: !16)
!23 = !DISubroutineType(types: !24)
!24 = !{!4, !3}
!25 = !DILocalVariable(name: "p", arg: 1, scope: !22, file: !1, line: 9, type: !3)
!26 = !DILocation(line: 9, column: 19, scope: !22)
!27 = !DILocalVariable(name: "tmp", scope: !22, file: !1, line: 10, type: !4)
!28 = !DILocation(line: 10, column: 7, scope: !22)
!29 = !DILocation(line: 10, column: 14, scope: !22)
!30 = !DILocation(line: 10, column: 13, scope: !22)
!31 = !DILocation(line: 11, column: 10, scope: !22)
!32 = !DILocation(line: 11, column: 9, scope: !22)
!33 = !DILocation(line: 11, column: 4, scope: !22)
!34 = !DILocation(line: 11, column: 6, scope: !22)
!35 = !DILocation(line: 12, column: 12, scope: !22)
!36 = !DILocation(line: 12, column: 11, scope: !22)
!37 = !DILocation(line: 12, column: 3, scope: !22)
!38 = distinct !DISubprogram(name: "main", scope: !1, file: !1, line: 15, type: !39, scopeLine: 15, spFlags: DISPFlagDefinition, unit: !0, retainedNodes: !16)
!39 = !DISubroutineType(types: !40)
!40 = !{!4}
!41 = !DILocalVariable(name: "r", scope: !38, file: !1, line: 16, type: !4)
!42 = !DILocation(line: 16, column: 7, scope: !38)
!43 = !DILocalVariable(name: "p", scope: !38, file: !1, line: 17, type: !3)
!44 = !DILocation(line: 17, column: 8, scope: !38)
!45 = !DILocation(line: 17, column: 20, scope: !38)
!46 = !DILocation(line: 18, column: 8, scope: !38)
!47 = !DILocation(line: 18, column: 3, scope: !38)
!48 = !DILocation(line: 19, column: 3, scope: !38)
!49 = !DILocation(line: 19, column: 11, scope: !38)
!50 = !DILocation(line: 19, column: 10, scope: !38)
!51 = !DILocation(line: 19, column: 13, scope: !38)
!52 = !DILocation(line: 20, column: 18, scope: !53)
!53 = distinct !DILexicalBlock(scope: !38, file: !1, line: 19, column: 20)
!54 = !DILocation(line: 20, column: 9, scope: !53)
!55 = !DILocation(line: 20, column: 7, scope: !53)
!56 = distinct !{!56, !48, !57, !58}
!57 = !DILocation(line: 21, column: 3, scope: !38)
!58 = !{!"llvm.loop.mustprogress"}
!59 = !DILocation(line: 22, column: 10, scope: !38)
!60 = !DILocation(line: 22, column: 3, scope: !38)
```

Source Code 2: Full LLVM IR source (`double.ll`) used for all sample animations.

Appendix B
Figures for Metrics



(a) Test coverage by module. Most modules exceed the 80% threshold; `render/stack_renderer` (48%) and `render/cfg_renderer` (62%) are the primary gaps due to Manim rendering dependencies.



(b) Test count per module (298 total). CLI, transform, and ingest modules carry the largest test loads.

Fig. 11: Test coverage and test count by module.

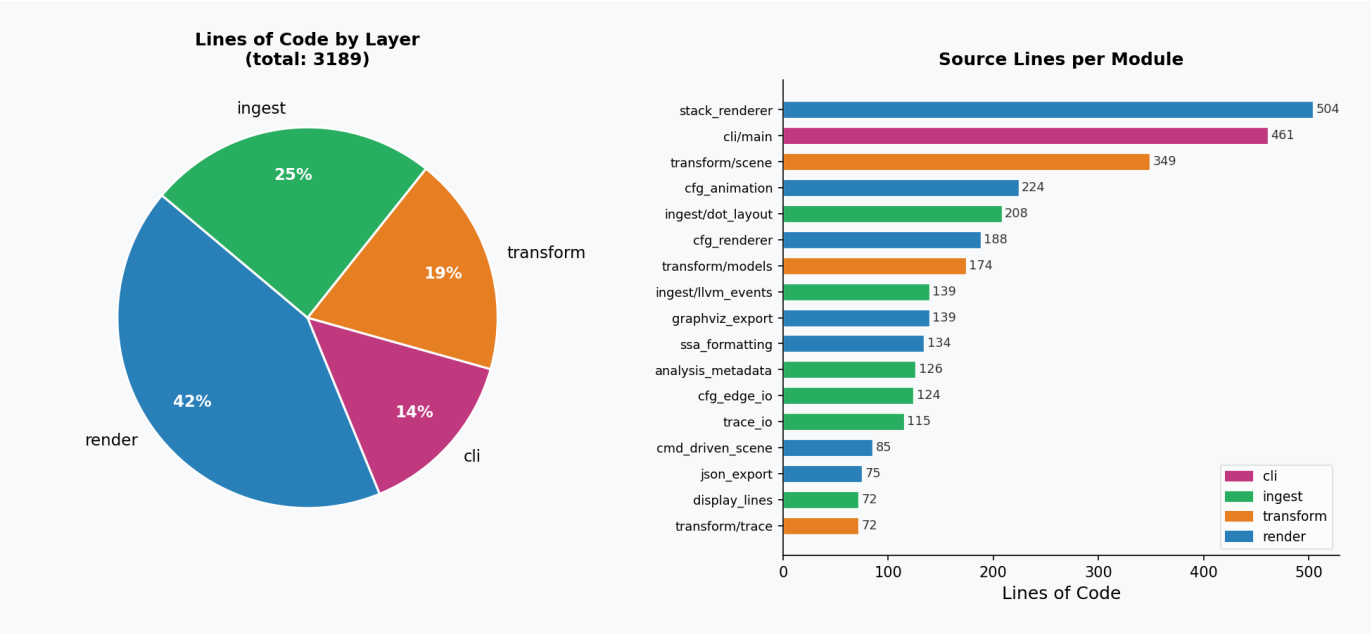


Fig. 12: Lines of code by layer (left) and per module (right). The Render layer is the largest due to the Manim animation logic in `stack_renderer` and `cfg_animation`.

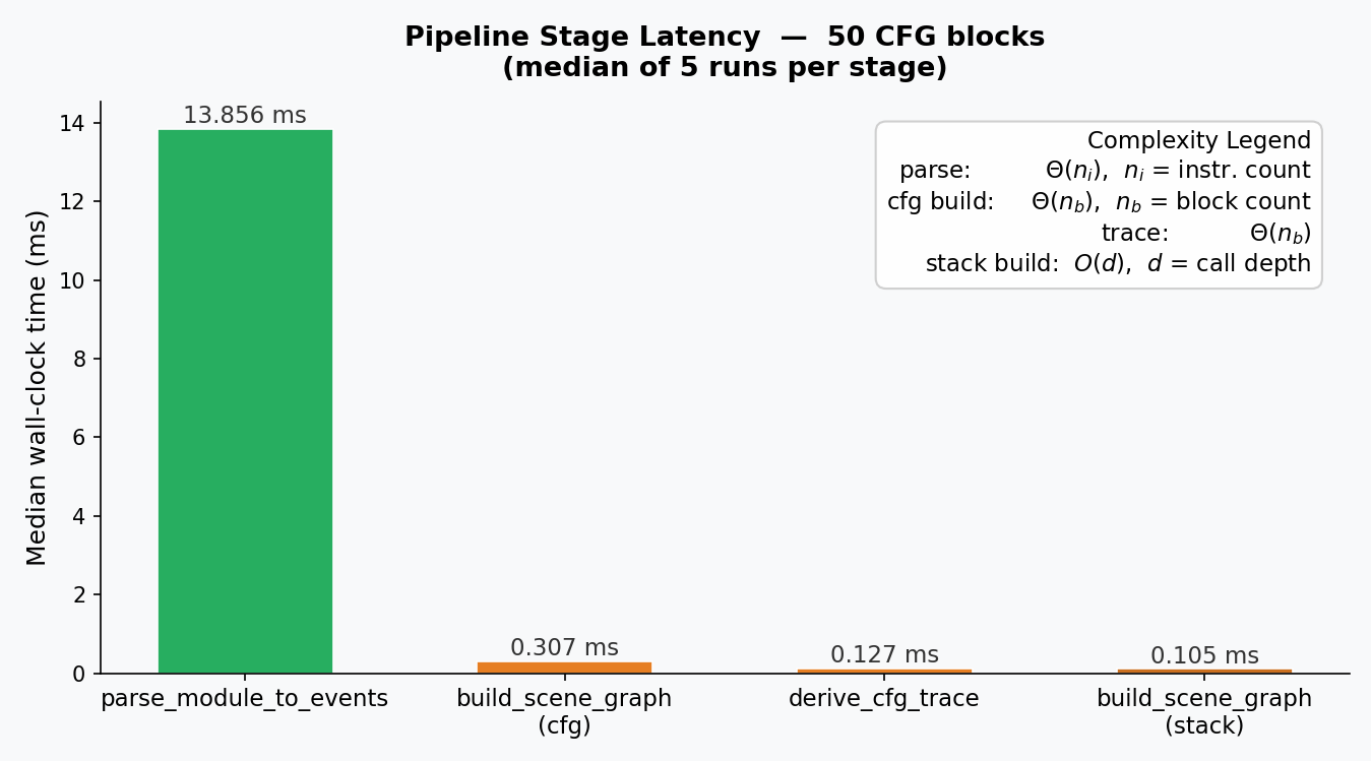


Fig. 13: Pipeline stage latency for a 50-block CFG (median of 5 runs). Parsing dominates; Transform stages are two orders of magnitude faster.

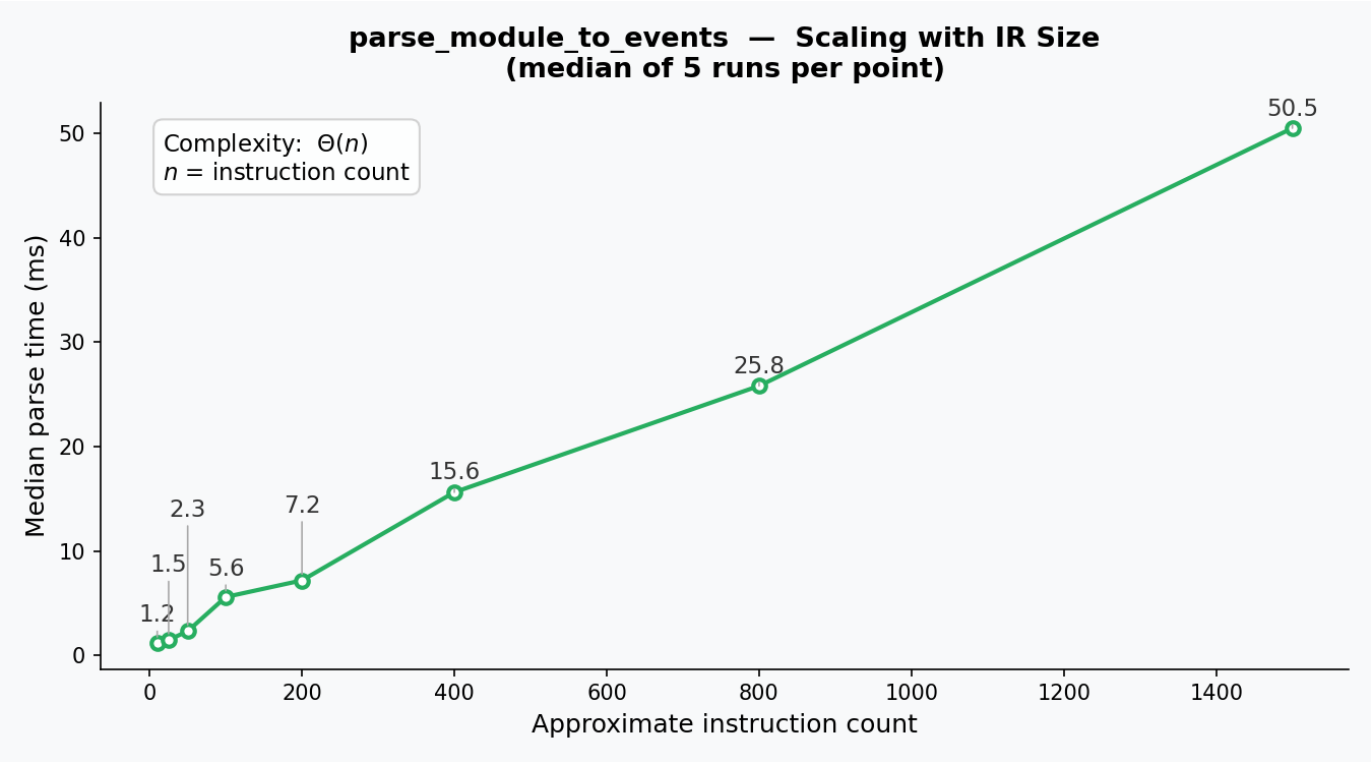


Fig. 14: `parse_module_to_events` scaling. Linear in instruction count—$\Theta(n)$.

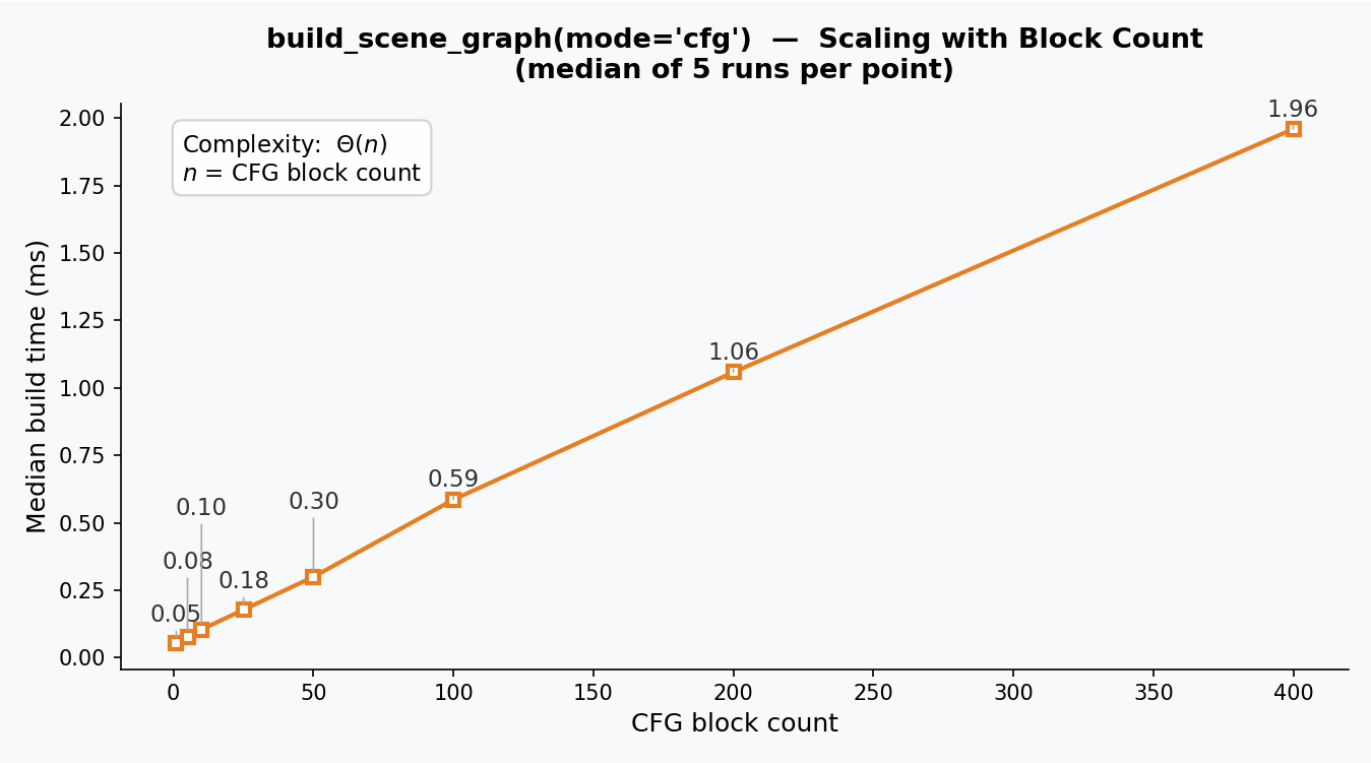


Fig. 15: CFG scene-graph build time vs. block count—$\Theta(n)$.

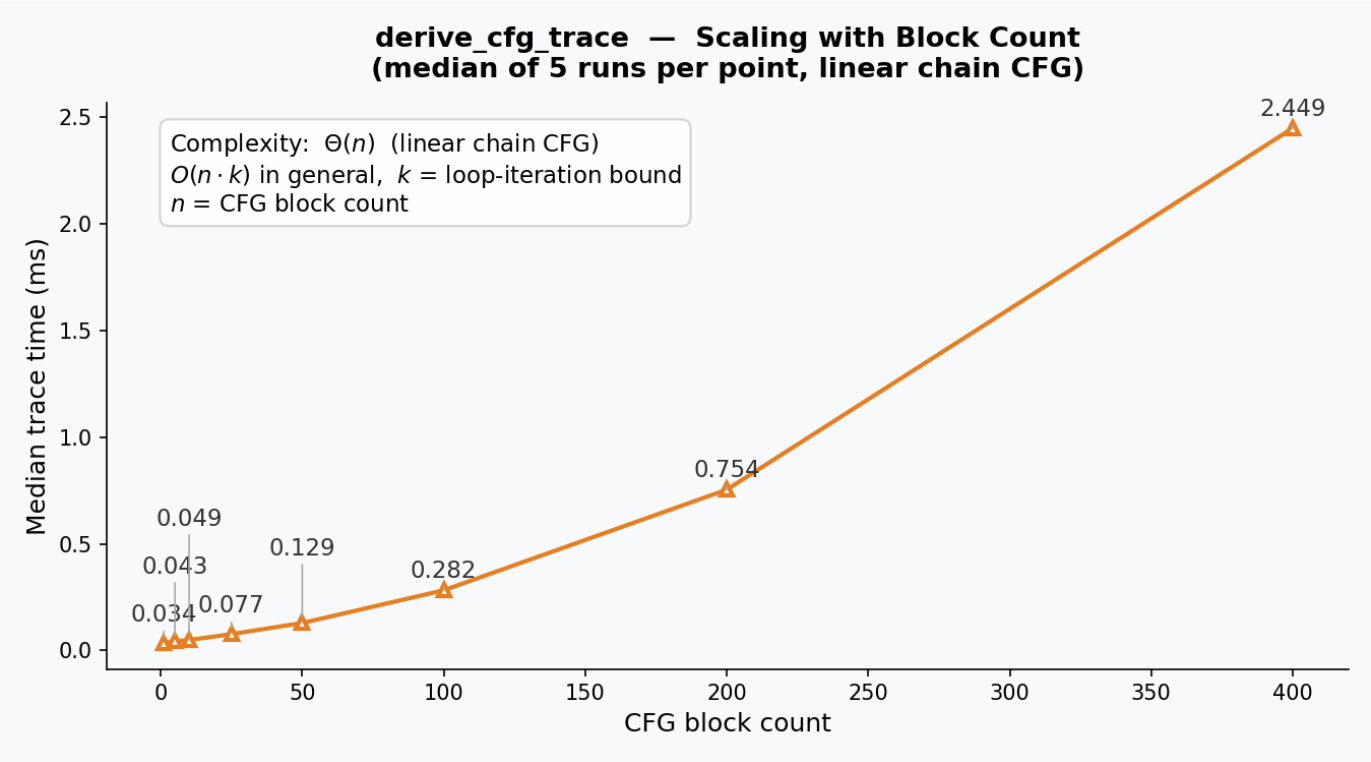


Fig. 16: CFG trace derivation scaling—$\Theta(n)$ for linear chains, $O(n \cdot k)$ with loops.

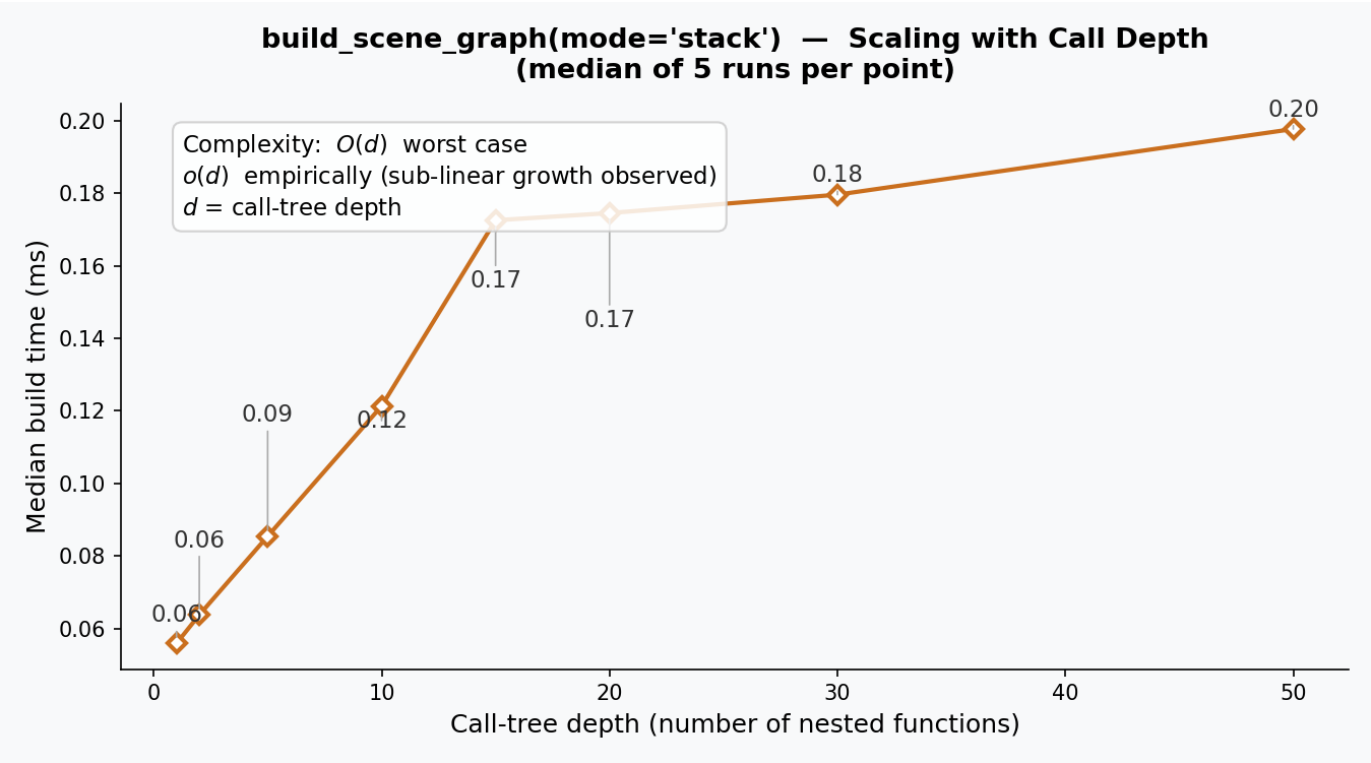


Fig. 17: Stack scene-graph build time vs. call depth—$O(d)$ worst case, sub-linear empirically.

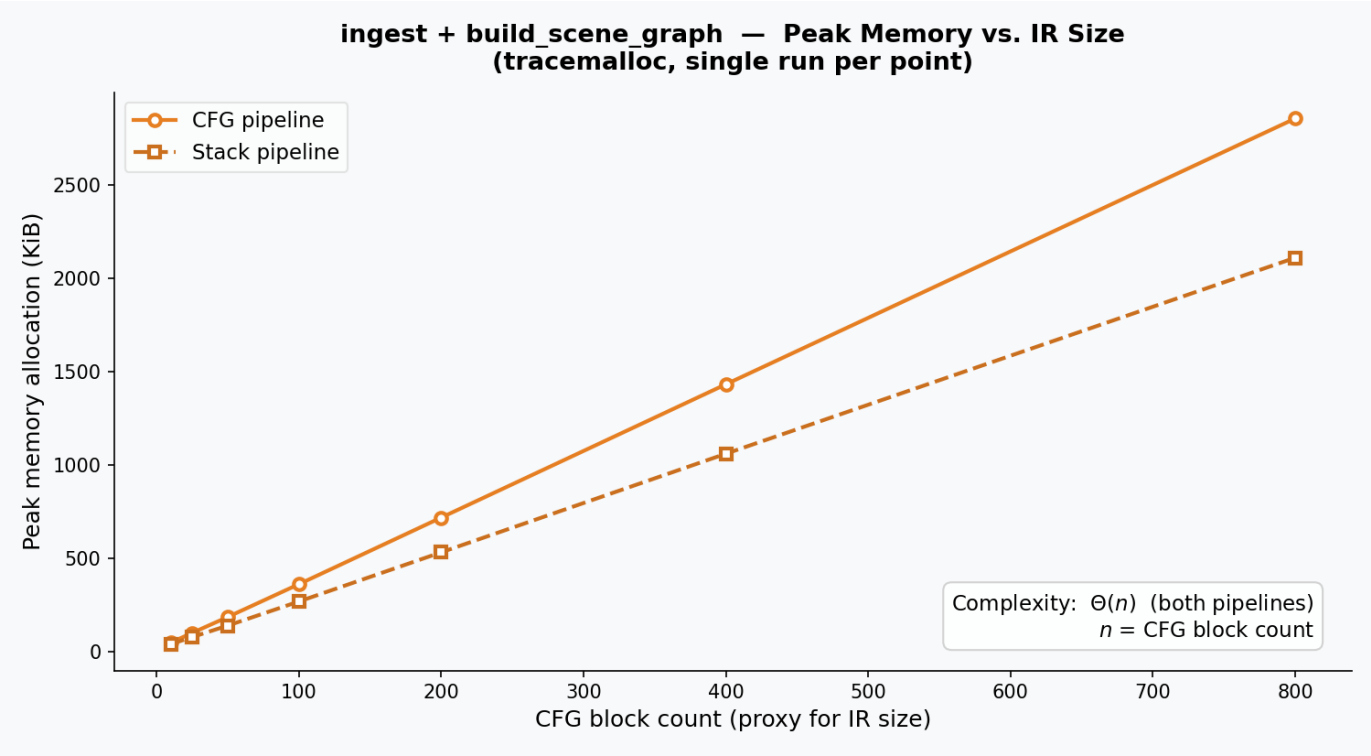


Fig. 18: Peak memory allocation vs. CFG block count. Both pipelines scale linearly—$\Theta(n)$.